

EduStack Masterclass

Your Ultimate Computer Science & Engineering Interview Preparation Hub

VOLUME 3 of 4 — MongoDB, Mongoose, Indexing, Caching & Cloud Storage

Database & Cloud: From Zero to FAANG Interview Ready

ACID | BSON | WiredTiger | Indexing | Aggregation | Mongoose | TTL Cache | Cloudinary | 40 Deep Q&As

Project:	EduStack - CS Student Resource Hub & AI Tutor Platform
This Volume:	MongoDB, Mongoose schemas, Indexing, Aggregation, Caching, Cloudinary, 40 Q&As
Volume 1:	JS Engine, Node.js Event Loop, Express Pipeline, REST API Design
Volume 2:	Authentication, JWT, bcrypt, Google OAuth, XSS, CSRF, Payment Security
Volume 4:	System Design, OS Concepts, DSA Patterns, Microservices, FAANG Scenarios
DB Stack:	MongoDB Atlas + Mongoose 8 + connect-mongodb-session + in-memory TTL cache
Media Stack:	Multer memoryStorage + Cloudinary CDN + bufferToBase64Uri
Indexes:	email (unique), googleId (index:true), { subject:1, type:1 } compound, TTL on OTP

Volume 3 of 4 | Read all 4 volumes to crack any backend/database interview at product-based companies

Table of Contents — Volume 3: Database & Cloud

1.	ACID Properties & Database Fundamentals Atomicity, Consistency, Isolation, Durability with concrete MongoDB examples
2.	SQL vs NoSQL - When to Use What Trade-offs, scaling patterns, when MongoDB is right and when it is not
3.	MongoDB Internals BSON storage, WiredTiger engine, journaling, write concerns, document model
4.	Mongoose Schema Design - EduStack Models User, Subject, Resource, Payment, OTP schemas with design rationale
5.	Indexing Deep Dive B-Tree structure, compound indexes, partial indexes, covered queries, explain()
6.	Mongoose Advanced Patterns populate(), \$lookup, findOneAndUpdate(), select(+password), pre-save hooks
7.	Aggregation Pipeline \$match, \$group, \$lookup, \$project, \$sort with real EduStack examples
8.	Caching Strategies TTL in-memory cache, DSA sheet 5-min cache, Redis patterns, LRU eviction
9.	MongoDB Session Store connect-mongodb-session, sessions collection, session vs JWT
10.	Cloud Media Pipeline multer memoryStorage, bufferToBase64Uri, Cloudinary upload, CDN architecture
11.	40 Deep Interview Q&As - Database & Cloud MongoDB, Mongoose, Indexing, ACID, Aggregation, Caching - FAANG level

SECTION 1

ACID Properties & Database Fundamentals

Atomicity, Consistency, Isolation, Durability - from first principles with MongoDB examples

1.1 ACID - The Four Pillars of Reliable Transactions

ACID is a set of four properties that guarantee database transactions are processed reliably. Understanding ACID is essential for database design interviews at any product-based company. These properties apply to both SQL databases (MySQL, PostgreSQL) and modern NoSQL databases with transaction support (MongoDB 4.0+ supports ACID transactions).

Property	What It Guarantees	Without It	MongoDB/EduStack Example
Atomicity	A transaction either FULLY completes or FULLY rolls back. No partial state.	Payment deducted from account but order not created - money lost without order.	User.create() either succeeds fully or fails - no half-created user documents.
Consistency	Transactions take the DB from one VALID state to another. Schema rules always maintained.	A user document missing required fields violates schema - inconsistent data.	Mongoose schema validation: { required: true } ensures consistency before save.
Isolation	Concurrent transactions do not see each other's intermediate state.	Two users simultaneously withdraw the last Rs.100 - both succeed (phantom read).	MongoDB default: document-level atomicity. Multi-document transactions for cross-collection consistency.
Durability	Once committed, a transaction persists even after system crashes.	Payment confirmed but DB crashes before writing to disk - data lost.	MongoDB uses journaling (WAL) - writes go to journal first, then to data files.

KEY CONCEPT: MongoDB before v4.0 only had document-level atomicity (a single document write is atomic). MongoDB 4.0+ added multi-document ACID transactions with `session.startTransaction()` - similar to SQL `BEGIN TRANSACTION`. EduStack does not currently use multi-document transactions but would need them for complex operations like "transfer premium status" involving multiple collections.

1.2 BASE vs ACID - Eventual Consistency

Many distributed NoSQL databases (Cassandra, DynamoDB) use the BASE model instead of ACID. BASE stands for: Basically Available, Soft state, Eventually consistent. This is a deliberate trade-off: sacrifice strong consistency for higher availability and partition tolerance.

	ACID (Strong Consistency)	BASE (Eventual Consistency)
Consistency	Immediate - all reads see latest committed write	Eventual - replicas may temporarily have stale data
Availability	May sacrifice availability during partition	Always available (may serve stale data during partition)
Transactions	Full multi-document transactions with rollback	Usually no cross-partition transactions
Use cases	Financial systems, order processing, auth	Social media feeds, analytics, product catalogs
Examples	PostgreSQL, MySQL, MongoDB with transactions	Cassandra, DynamoDB, CouchDB, ElasticSearch

SQL vs NoSQL - Decision Framework

When to use relational vs document databases - trade-offs at scale

2.1 The Core Difference

- **SQL (Relational):** Data stored in tables with rows and columns. Fixed schema defined upfront. Relations through foreign keys and JOINS. ACID transactions natively. Query language: SQL. Examples: PostgreSQL, MySQL, SQLite.
- **NoSQL (Document):** Data stored as documents (JSON-like BSON). Flexible schema - each document can have different fields. Denormalization preferred over JOINS. Horizontal scaling native. Examples: MongoDB, CouchDB.
- **MongoDB Document Model:** A "subject" with its linked "resources" can be queried in ONE collection scan with \$lookup, or embedded directly. This is fundamentally different from SQL where you need JOIN across two tables.
- **EduStack uses MongoDB:** Educational content (subjects, resources) has flexible, evolving schemas. The platform needs to store varied resource types (notes, PYQs, playlists, links) with different optional fields. Document model fits this naturally.

Factor	Choose SQL When...	Choose MongoDB When...
Data Structure	Data is highly structured, rarely changes shape, has strong relationships	Data shape varies per document, evolves frequently, or is hierarchical
Relationships	Many complex many-to-many relationships that need JOINS	One-to-many or embedded relationships (subject has many resources)
Consistency	Strong ACID required across multiple tables (banking, payments)	Eventual consistency acceptable, or single-collection operations sufficient
Scalability	Vertical scaling OK, or sharding is complex	Horizontal sharding required (write-heavy, massive scale)
EduStack fit	Payments table has fixed schema (amount, status, orderId)	Subjects/Resources have flexible types, optional fields, embeddings

MongoDB Internals - BSON, WiredTiger & Journaling

How MongoDB actually stores data - from disk to memory

3.1 BSON - Binary JSON

MongoDB stores documents in BSON (Binary JSON), not plain JSON text. BSON extends JSON with additional data types and is designed for efficient traversal, encoding, and decoding. BSON is binary - it is NOT human-readable but is significantly faster to parse than text JSON.

BSON Type	Size	Not in JSON?	Usage in EduStack
ObjectId	12 bytes	Yes - MongoDB's unique ID type	Every document's <code>_id</code> , and all ref: "User" / "Subject" fields
Date	8 bytes (64-bit int)	JSON has no Date - would serialize as string	<code>createdAt</code> , <code>updatedAt</code> (timestamps), <code>expiresAt</code> in OTP
Int32/Int64	4/8 bytes	JSON number has no int/float distinction	semester (1-8), views counter, amount in paise
Boolean	1 byte	JSON has boolean but BSON is explicit	<code>isVerified</code> , <code>isPremium</code> , <code>httpOnly</code>
Binary	Variable	JSON has no binary type	File data, bcrypt hashes (stored as string in EduStack)

3.2 WiredTiger Storage Engine

WiredTiger is MongoDB's default storage engine (since v3.2). It provides document-level concurrency control, compression, and journaling. Understanding WiredTiger is important for database performance tuning questions.

- **Document-level locking:** Multiple operations can simultaneously modify DIFFERENT documents. No collection-level or database-level write locks (unlike older MMAPv1 engine). Enables high write concurrency.
- **Compression:** WiredTiger compresses data on disk using Snappy (default, fast) or zlib (better compression, slower). EduStack's Atlas cluster uses Snappy compression by default.
- **Cache:** WiredTiger maintains an in-memory cache (default: 50% of RAM or 256MB, whichever is larger). Frequently accessed documents (hot data) stay in cache for fast reads without disk I/O.
- **Journaling (Write-Ahead Log):** All writes go to the journal FIRST, then to data files. If the server crashes, MongoDB replays the journal on restart to recover committed operations. Ensures Durability (the D in ACID).
- **Checkpoints:** WiredTiger periodically flushes dirty pages (modified data) from cache to disk files. Checkpoint interval: 60 seconds by default. After a crash, MongoDB replays only the journal since the last checkpoint.

3.3 Write Concerns & Read Preferences

Write Concern	What It Means	When to Use
<code>{ w: 0 }</code>	Fire-and-forget - no acknowledgment from MongoDB	Low-importance logs, analytics where some loss is acceptable
<code>{ w: 1 }</code>	Primary node acknowledges the write (default in MongoDB)	Standard CRUD operations. Fast, but data can be lost if primary crashes before replication
<code>{ w: "majority" }</code>	Majority of replica set members acknowledge. Survives primary failure.	Critical data: user registrations, payments. EduStack should use this in production.
<code>{ w: 1, j: true }</code>	Primary acknowledges AND journal write is confirmed	Important operations requiring durability guarantee

Mongoose Schema Design - All EduStack Models

User, Subject, Resource, Payment, OTP schemas with full design rationale

4.1 User Schema - Design Deep Dive

JavaScript / Mongoose

```
// models/user.js - Full schema with design annotations
const userSchema = new mongoose.Schema({
  // IDENTITY
  firstName: { type: String, required: true, trim: true, maxlength: 50 },
  lastName: { type: String, required: true, trim: true, maxlength: 50 },

  // Email: unique index + lowercase normalization
  // lowercase: true transforms before save - "USER@TEST.COM" -> "user@test.com"
  // unique: true creates a unique B-Tree index on this field
  email: {
    type: String, required: true, unique: true,
    lowercase: true, trim: true,
    match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, "Invalid email format"],
  },

  // AUTHENTICATION
  // select: false = NEVER returned in queries unless .select("+password") used
  // minlength: 6 = Mongoose-level validation (not bcrypt - bcrypt has no minimum)
  password: { type: String, select: false, minlength: 6 },

  // index: true = O(log n) lookup on every Google OAuth login (prevents full scan)
  googleId: { type: String, default: null, index: true },

  // ROLE & STATUS
  role: { type: String, enum: ["user", "student", "contributor", "admin"], default: "user" },
  isVerified: { type: Boolean, default: false }, // false until OTP verified
  isPremium: { type: Boolean, default: false }, // true after Razorpay payment

  // PROFILE
  avatar: { type: String, default: "default-avatar.png" },
  phoneNumber: { type: String, trim: true, default: null },
  bio: { type: String, maxlength: 300, default: "" },
  branch: { type: String, trim: true, default: "CSE" },
  semester: { type: Number, min: 1, max: 8, default: 1 },
}, { timestamps: true }); // Auto-adds createdAt, updatedAt
```

JavaScript / Mongoose

```
// Virtual field - computed, NOT stored in MongoDB
userSchema.virtual("fullName").get(function() {
  return `${this.firstName} ${this.lastName}`;
});

// Singleton pattern - prevents model recompilation in hot-reload
const User = mongoose.models.User || mongoose.model("User", userSchema);
```

4.2 Resource Schema - Compound Index

```
// models/resource.js - Key design decisions
const resourceSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true, maxlength: 150 },
  description: { type: String, trim: true, maxlength: 500, default: "" },

  // Discriminator field - tells frontend HOW to render this resource
  type: {
    type: String,
    enum: ["note", "pyq", "playlist", "link", "platform"],
    required: true,
  },

  url: { type: String, required: true, trim: true }, // External URL only - no file uploads

  // Relationships - ObjectId refs for population
  subject: { type: mongoose.Schema.Types.ObjectId, ref: "Subject", required: true },
  uploadedBy: { type: mongoose.Schema.Types.ObjectId, ref: "User" },

  isPremium: { type: Boolean, default: false }, // Premium gate per resource
  views: { type: Number, default: 0, min: 0 }, // Simple view counter
}, { timestamps: true });

// COMPOUND INDEX: Optimizes "all notes for subject X" query pattern
// Without this: MongoDB scans ALL resources for subject + type match
// With this: MongoDB looks up the index directly - O(log n) not O(n)
resourceSchema.index({ subject: 1, type: 1 });
```

FAANG TIP: Why a compound index { subject: 1, type: 1 } instead of two separate single-field indexes? MongoDB can only use ONE index per query. A compound index covers BOTH conditions efficiently. Also, the LEFT-PREFIX RULE: { subject: 1, type: 1 } also optimizes queries filtering ONLY by subject (because subject is the leading key), but NOT queries filtering only by type.

SECTION 5

Indexing Deep Dive

B-Tree structure, compound indexes, covered queries, explain() - from first principles

5.1 How Indexes Work - B-Tree Structure

A database index is a separate data structure (separate from the collection) that MongoDB maintains to support fast lookup of documents. Without an index, MongoDB must perform a "collection scan" - reading every document to find matches. With an index, MongoDB can jump directly to the matching location.

MongoDB uses B-Tree (Balanced Tree) indexes. A B-Tree keeps data sorted and provides $O(\log n)$ time complexity for search, insert, and delete. For a collection of 10 million documents, finding a user by email takes about 23 comparisons with a B-Tree index vs 10 million reads without.

Index Type	What It Does	EduStack Usage	Performance Impact
Single Field	B-Tree on one field. Supports equality, range, sort queries.	googleId: true, email: unique:true	$O(\log n)$ lookup vs $O(n)$ collection scan
Compound	B-Tree on multiple fields in order. Left-prefix rule.	{ subject: 1, type: 1 } on Resource	Covers multi-field queries with single index lookup
Unique	Single-field + uniqueness constraint. MongoDB rejects duplicates.	email: { unique: true }	Prevents duplicate inserts at DB level + $O(\log n)$ lookup
TTL (Time-To-Live)	Background worker deletes documents when expiresAt is in the past.	expiresAt on OTP collection	Automatic cleanup - no cron job needed
Text Index	Full-text search on string fields. Supports \$text: { \$search: "..." }.	Not yet used - could index subject.name for search	Enables full-text search on collection

5.2 The Left-Prefix Rule for Compound Indexes

JavaScript / Mongoose

```
// Compound index: { subject: 1, type: 1 }
// Index keys are stored sorted: first by subject, then by type within same subject

// QUERY 1: Filter by BOTH subject AND type -> USES INDEX (both fields covered)
Resource.find({ subject: subjectId, type: "note" });

// QUERY 2: Filter by ONLY subject (left prefix) -> USES INDEX (partial use)
Resource.find({ subject: subjectId });

// QUERY 3: Filter by ONLY type (right field, no left prefix) -> COLLECTION SCAN
Resource.find({ type: "note" }); // Cannot use { subject:1, type:1 } alone!
// For this query, you would need a separate { type: 1 } index

// Verify with explain():
const explanation = await Resource.find({ subject: subjectId, type: "note" })
  .explain("executionStats");
console.log(explanation.queryPlanner.winningPlan.inputStage.indexName);
// Expected: "subject_1_type_1"
console.log(explanation.executionStats.totalDocsExamined);
// With index: small number. Without index: = total documents in collection
```

5.3 Covered Queries - Zero Document Reads

A "covered query" is one where ALL data needed for the response is available IN the index itself - MongoDB never needs to load the actual documents from disk. This is the fastest possible query type.

JavaScript / Mongoose

```
// Covered query example:
// Index: { subject: 1, type: 1 }
// Query: find by subject + type, return ONLY subject and type fields
const types = await Resource.find(
  { subject: subjectId, type: "note" }, // Filter
  { subject: 1, type: 1, _id: 0 }       // Projection: only indexed fields
);
// MongoDB reads from index only - ZERO document reads from disk
// explain() will show: "IXSCAN" with no "FETCH" stage

// NOT a covered query:
Resource.find({ subject: subjectId }, { title: 1 });
// "title" is NOT in the index - MongoDB must fetch the document
// explain() shows: "IXSCAN" -> "FETCH" (additional disk read per document)
```

Mongoose Advanced Patterns

populate(), \$lookup, findOneAndUpdate(), select(+password), pre-save hooks

6.1 populate() vs \$lookup - When to Use Each

Both populate() and \$lookup join data from different collections. populate() is Mongoose-specific and issues a separate query for each referenced document. \$lookup is MongoDB's native aggregation JOIN that retrieves related documents in a single pipeline stage.

JavaScript / Mongoose

```
// populate() - Mongoose convenience method (runs separate query)
// Gets all resources and populates the "subject" and "uploadedBy" fields
const resources = await Resource.find({ type: "note" })
  .populate("subject", "name code semester") // Only select name, code, semester from Subject
  .populate("uploadedBy", "firstName email"); // Only select firstName, email from User

// Under the hood: 3 queries:
// 1. Resource.find({ type: "note" }) -> array of resources
// 2. Subject.find({ _id: { $in: [subjectIds...] } }) -> subjects
// 3. User.find({ _id: { $in: [userIds...] } }) -> users
// Then Mongoose merges the results in JavaScript memory

// $lookup - Native MongoDB aggregation JOIN (single pipeline)
const resources = await Resource.aggregate([
  { $match: { type: "note" } },
  { $lookup: {
    from: "subjects", // Collection name (lowercase plural)
    localField: "subject", // Field in Resource (ObjectId)
    foreignField: "_id", // Field in Subject
    as: "subjectData", // Output array field name
  }},
  { $unwind: "$subjectData" }, // Flatten array to single object
]);

// Use populate() for simple lookups in application code (cleaner)
// Use $lookup in aggregation pipelines where you need $group, $sort, $count etc.
```

6.2 findOneAndUpdate() - Atomic Read-Modify-Write

JavaScript / Mongoose

```
// findOneAndUpdate() atomically finds + updates in single MongoDB operation
// Returns EITHER the old document OR the new document based on { new: } option

// 1. Mark user as verified after OTP check:
const user = await User.findOneAndUpdate(
  { email: normalizedEmail },      // Filter
  { isVerified: true },            // Update
  { new: true }                    // Return the UPDATED document (not the old one)
);

// 2. Grant premium after payment:
await User.findByIdAndUpdate(req.user._id, { isPremium: true });
// findByIdAndUpdate is shorthand for findOneAndUpdate({ _id: id }, ...)

// 3. Update payment status:
await Payment.findOneAndUpdate(
  { razorpayOrderId },              // Find by Razorpay order ID
  { status: "paid", razorpayPaymentId, razorpaySignature }, // Update multiple fields
);

// 4. Upsert OTP (create or update):
await OTP.findOneAndUpdate(
  { email },                        // Filter
  { email, code, expiresAt },       // Update/set fields
  { upsert: true, new: true }        // CREATE if not found + return new doc
);

// WHY NOT find() then save()?
// const user = await User.findOne({ email });
// user.isVerified = true;
// await user.save();
// This is TWO operations - a race condition can occur between find and save.
// findOneAndUpdate is ATOMIC - safe for concurrent requests.
```

6.3 Mongoose Middleware (Hooks)

- **Pre hooks:** Run before a model operation. Used in EduStack to hash passwords before save: `userSchema.pre("save", async function() { ... })`. The function MUST be a regular function (not arrow) so "this" = document instance.
- **Post hooks:** Run after an operation. Useful for sending emails after user creation, triggering side effects.
- **isModified():** `userSchema.pre("save")` checks `if(!this.isModified("password")) return next()` - prevents re-hashing an already-hashed password when saving unrelated fields (e.g., updating avatar).
- **pre("validate"):** Runs before Mongoose validation. Useful for transforming data before validation rules are checked.
- **Query middleware:** `pre("find")`, `pre("findOne")` - run before queries. Can add default filters (e.g., always exclude soft-deleted documents).

MongoDB Aggregation Pipeline

\$match, \$group, \$lookup, \$project, \$sort - from first principles

7.1 What is the Aggregation Pipeline?

MongoDB's aggregation pipeline is a data processing framework that transforms documents through a sequence of stages. Each stage takes the output of the previous stage as input. This is similar to Unix pipes: data flows through a series of transformations.

JavaScript / Mongoose

```
// Aggregation Pipeline Example: Count resources per subject, sorted by count
const resourceStats = await Resource.aggregate([

  // STAGE 1: $match - Filter documents (like WHERE in SQL)
  { $match: { isPremium: false } }, // Only count free resources

  // STAGE 2: $group - Group by field, compute aggregates
  { $group: {
    _id: "$subject", // Group by subject ObjectId
    totalResources: { $sum: 1 }, // Count documents per group
    typeBreakdown: { $push: "$type" }, // Collect all types into array
    avgViews: { $avg: "$views" }, // Average views per subject
  }},

  // STAGE 3: $lookup - JOIN with subjects collection
  { $lookup: {
    from: "subjects",
    localField: "_id", // _id here is the subject ObjectId from $group
    foreignField: "_id",
    as: "subjectInfo",
  }},

  // STAGE 4: $unwind - Deconstruct subjectInfo array (each doc has one subject)
  { $unwind: "$subjectInfo" },

  // STAGE 5: $project - Shape the output (like SELECT in SQL)
  { $project: {
    subjectName: "$subjectInfo.name",
    subjectCode: "$subjectInfo.code",
    totalResources: 1,
    avgViews: { $round: ["$avgViews", 2] }, // Round to 2 decimal places
    _id: 0, // Exclude _id from output
  }},

  // STAGE 6: $sort - Sort output
  { $sort: { totalResources: -1 } }, // Descending: most resources first

]);
```

JavaScript / Mongoose

```
// STAGE 7: $limit - Pagination
{ $limit: 10 }, // Top 10 subjects by resource count
]);

// Aggregation NEVER returns Mongoose Documents - always plain JS objects
// Schema validation and virtuals do NOT apply to aggregation output
```

FAANG TIP: Aggregation pipeline key insight: Order of stages matters for performance. Always \$match FIRST to reduce the number of documents flowing through subsequent stages. Placing \$match after \$lookup means MongoDB joins all documents first, then filters - much slower. MongoDB can use indexes for \$match placed at the pipeline beginning.

Caching Strategies

EduStack's TTL in-memory cache, Redis patterns, LRU eviction - from first principles

8.1 Why Cache? The Performance Problem

EduStack's DSA sheet contains 450+ problems sourced from a public Google Sheet CSV. Without caching, every request to /api/dsa-sheet would fetch and parse the entire CSV from Google Sheets (network I/O + CSV parsing). At 100 requests/minute, this would hammer Google's servers and slow every user request to 500ms+.

EduStack's In-Memory TTL Cache Implementation

JavaScript / Mongoose

```
// app.js - In-memory TTL cache for DSA sheet (real EduStack code)

let _dsaSheetCache = null; // Stores parsed problems array
let _dsaSheetCacheTime = 0; // Last successful fetch timestamp
const DSA_CACHE_TTL_MS = 5 * 60 * 1000; // 5 minutes TTL

app.get("/api/dsa-sheet/sync", async (req, res) => {
  const bustCache = req.query.bust || false;

  // Check if cache is FRESH (within 5 minutes)
  const isFresh = _dsaSheetCache && (Date.now() - _dsaSheetCacheTime < DSA_CACHE_TTL_MS);

  if (isFresh && !bustCache) {
    // CACHE HIT: Return immediately without any I/O
    return res.json({ success: true, source: "cache", data: _dsaSheetCache });
  }

  // CACHE MISS or BUST: Fetch live from Google Sheets
  try {
    const csvText = await fetchCSV(GOOGLE_SHEET_CSV_URL); // HTTP request
    const liveProblems = csvLinesToProblems(parseCSVText(csvText));

    if (liveProblems && liveProblems.length > 5) {
      _dsaSheetCache = liveProblems; // Update in-memory cache
      _dsaSheetCacheTime = Date.now(); // Reset TTL timer
      fs.writeFileSync(jsonPath, JSON.stringify(liveProblems, null, 2)); // Persist to disk
      return res.json({ success: true, source: "live", data: liveProblems });
    }
  } catch (err) {
    // Fallback chain: in-memory stale -> disk file -> empty array
    if (_dsaSheetCache) return res.json({ source: "stale-cache", data: _dsaSheetCache });
    const diskData = JSON.parse(fs.readFileSync(jsonPath, "utf-8"));
    return res.json({ source: "disk-fallback", data: diskData });
  }
});
```

8.2 Cache Patterns - Beyond Simple TTL

Pattern	How It Works	Use Case	EduStack Equivalent
TTL Cache	Expire cache entry after N seconds. Re-fetch on next request.	Data that changes periodically (DSA sheet updates hourly)	DSA sheet: 5-minute TTL cache
LRU Cache	Least Recently Used eviction. When cache is full, remove the item accessed longest ago.	Many objects, limited memory - keep hot data in cache	Not used - only one cached object (DSA sheet)

Cache-Aside	App checks cache first. Cache miss -> load from DB -> write to cache -> return.	DB query results: user profiles, subject lists	Could cache User.findById(id) results
Write-Through	Write to cache AND DB simultaneously. Cache always has latest.	Write-heavy systems needing read speed	Could use when updating payment status
Read-Through	Cache handles DB misses automatically. App only talks to cache.	Transparent caching layer (Redis, Memcached as cache layer)	Redis caching layer in production scale

8.3 Redis for Production Caching

- Redis is an in-memory key-value store used as a distributed cache. Unlike EduStack's single-process memory cache, Redis survives process restarts and is shared across multiple Node.js instances (cluster mode or multiple servers).
- **Why Redis over in-memory for production:** EduStack on Render.com's free tier restarts periodically (cold starts). The DSA cache resets on every restart. Redis (persistent or with replication) would retain cache across restarts.
- **Redis data structures:** Strings (simple key-value), Lists (queues), Sets, Sorted Sets (leaderboards), Hashes (object-like), Pub/Sub (real-time messaging). All stored in RAM for microsecond latency.
- **Redis patterns in Node.js:** `const client = redis.createClient(); await client.setEx("dsa_cache", 300, JSON.stringify(data)); const cached = await client.get("dsa_cache"); if (cached) return JSON.parse(cached);`
- **Eviction policies:** allkeys-lru (remove least recently used when memory full), volatile-ttl (remove keys with shortest TTL), noeviction (return error when full). Configure based on your caching needs.

MongoDB Session Store

connect-mongodb-session, the sessions collection, session vs JWT comparison

9.1 How Sessions Work in EduStack

EduStack uses express-session with connect-mongodb-session as the storage adapter. This creates a "sessions" collection in MongoDB where session data is persisted. Sessions are used exclusively for Google OAuth flow - regular email/password auth uses only JWT cookies.

JavaScript / Mongoose

```
// app.js - MongoDB Session Store Configuration
const MongoDBStore = require("connect-mongodb-session")(session);

const store = new MongoDBStore({
  uri: process.env.MONGO_URI, // Same MongoDB Atlas cluster
  collection: "sessions",      // Stores sessions in a "sessions" collection
});

store.on("error", function(error) {
  console.error("Session Store Error:", error); // Log but don't crash
});

app.use(session({
  secret: process.env.JWT_SECRET, // Used to sign session ID cookie
  resave: false,                  // Don't save session if unmodified
  saveUninitialized: false,       // Don't create session until data is stored
  cookie: {
    secure: IS_PRODUCTION,        // HTTPS-only in production
    httpOnly: true,               // JS cannot read session cookie
    sameSite: IS_PRODUCTION ? "none" : "lax",
    maxAge: 7 * 24 * 60 * 60 * 1000, // 7 days
  },
  store: store, // Persist sessions to MongoDB (not in-memory)
}));

// Passport uses the session to store user._id between OAuth redirects:
passport.serializeUser((user, done) => done(null, user._id));
passport.deserializeUser(async (id, done) => {
  const user = await User.findById(id);
  done(null, user);
});
```

KEY CONCEPT: The "sessions" collection in MongoDB stores: `_id` (session ID), `session` (serialized session data including `passport.user = user._id`), `expires` (TTL for auto-cleanup). MongoDB automatically cleans up expired sessions. Session ID is a cryptographically random string, not the user ID - it is safe to store in a cookie.

Cloud Media Pipeline - Multer + Cloudinary

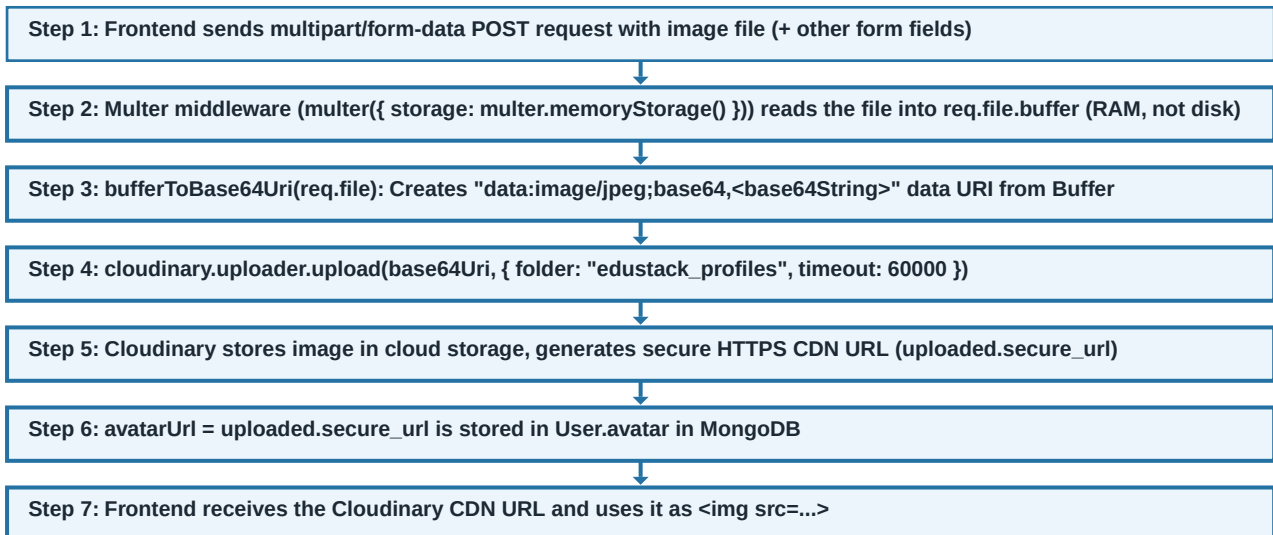
memoryStorage, bufferToBase64Uri, Cloudinary upload, CDN architecture

10.1 Why Not Store Files on the Server?

- **Ephemeral filesystem:** Render.com (and most PaaS platforms) uses an ephemeral filesystem. Files written to disk are LOST when the process restarts or the container is recreated. A deploy wipes all local files.
- **Stateless requirement:** Running multiple Node.js instances (horizontal scaling) means different instances have different local files. Request 1 to server-A uploads avatar. Request 2 to server-B asks for the avatar - server-B has no file.
- **CDN advantages:** Cloudinary serves files from a global CDN with edge locations. Users in India get images from the nearest Cloudinary edge node, not from your Render.com server. Much faster image delivery.
- **Storage scalability:** Cloudinary has effectively unlimited storage (paid). Local disk is limited to the server's volume size.

10.2 The Full Upload Pipeline

FLOW: EduStack Avatar Upload Flow




```
// config/cloudinary.js - Cloudinary configuration
const { v2: cloudinary } = require("cloudinary");

cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME, // From .env
  api_key: process.env.CLOUDINARY_API_KEY, // From .env
  api_secret: process.env.CLOUDINARY_API_SECRET, // From .env - SECRET
});

// Convert multer Buffer to Cloudinary-compatible data URI
const bufferToBase64Uri = (file) => {
  const b64 = Buffer.from(file.buffer).toString("base64");
  const mimeType = file.mimetype; // e.g., "image/jpeg", "image/png"
  return `data:${mimeType};base64,${b64}`;
};

// authController.js - Avatar upload with fallback:
if (req.file) {
  try {
    const base64Uri = bufferToBase64Uri(req.file);
    const uploaded = await cloudinary.uploader.upload(base64Uri, {
      folder: "edustack_profiles",
      timeout: 60000, // 60 second timeout for large uploads
    });
    avatarUrl = uploaded.secure_url; // HTTPS Cloudinary CDN URL
  } catch (cloudErr) {
    console.warn("Cloudinary upload failed:", cloudErr.message);
    // Fallback: store base64 directly in DB (large document - not ideal for prod)
    avatarUrl = `data:${req.file.mimetype};base64,${req.file.buffer.toString("base64")}`;
  }
}
```

40 Deep Interview Q&As - Database & Cloud

MongoDB, Mongoose, Indexing, ACID, Aggregation, Caching - FAANG level

About This Section: These 40 questions cover databases, MongoDB internals, Mongoose ODM patterns, indexing, and cloud storage at FAANG/Tier-1 interview depth. All answers reference EduStack's actual production database design.

Q1: What is MongoDB? How is it different from SQL databases?

Answer:

MongoDB is a document-oriented NoSQL database. Data is stored as BSON (Binary JSON) documents in collections (analogous to SQL tables). Documents in the same collection can have different fields - there is no rigid schema enforced at the database level (Mongoose adds schema validation at the application level).

- > SQL: Rows in tables with fixed columns, JOINS for relationships, ACID by default, strong consistency.
- > MongoDB: Documents in collections with flexible structure, \$lookup for joins (or embed related data), ACID per document (multi-document ACID via transactions in v4.0+).
- > EduStack uses MongoDB: Educational content schemas evolve frequently, different resource types have different optional fields, and Atlas provides managed hosting with automatic failover.

Q2: Explain ACID properties with a concrete example.

Answer:

ACID ensures database transactions are reliable. Example: User pays Rs.5 for premium access. The transaction must: (A) Atomically mark payment as "paid" AND set isPremium:true - both or neither. (C) Keep user schema consistent - isPremium must be boolean, not string. (I) Two concurrent payment verifications don't see each other's partial state. (D) Once isPremium is set true, it persists even if the server crashes immediately after.

- > MongoDB document-level atomic writes: Updating { isPremium: true } on a single User document is atomic - it either fully completes or doesn't.
- > Multi-document ACID: The Payment status update AND User.isPremium update are in different collections. For true ACID across both, a MongoDB session.startTransaction() would be needed.
- > EduStack's current design: If server crashes between Payment update and User.isPremium update, the payment is marked "paid" but premium not granted - a known gap that could be fixed with transactions.

Q3: What is a Mongoose schema? Why use Mongoose instead of raw MongoDB driver?

Answer:

A Mongoose schema defines the structure, types, defaults, validators, and indexes for documents in a collection. Mongoose provides: schema validation before saves, middleware hooks (pre/post), virtuals (computed properties), method definitions, population (reference resolution), and a cleaner query API.

- > Raw MongoDB driver: `const coll = db.collection("users"); await coll.insertOne({...})` - no validation, no hooks, no schema.
- > Mongoose: `const User = mongoose.model("User", userSchema); await User.create({...})` - validates against schema, runs pre-save hooks, enforces types and constraints.
- > EduStack uses Mongoose 8 for all database operations - schema validation catches data errors early (at app layer) before they reach MongoDB.

Q4: Explain Mongoose's unique index. How does it differ from unique: true in schema?

Answer:

In Mongoose, `unique: true` in the schema field definition creates a MongoDB unique index on that field. It does NOT add Mongoose validation - it relies on MongoDB throwing an E11000 error on duplicate insert. The difference: Mongoose validation fires before save (returns `ValidationError`), while unique constraint fires at DB level (returns `MongoServerError` with code 11000).

- > This means Mongoose does not check uniqueness before inserting - it attempts the insert and catches the DB error if duplicate.
- > The correct way to handle duplicate email: Use `User.findOne({ email })` before `User.create()` for a user-friendly error message. Rely on E11000 as the safety net (in case of race condition between `findOne` and `create`).
- > EduStack's errorHandler maps `err.code === 11000` to HTTP 409 Conflict.

Q5: What is the difference between `.save()` and `.create()` in Mongoose?

Answer:

`.create()` is a static method that creates a new document and saves it in one call. It is equivalent to `new Model(data).save()`. `.save()` is an instance method on a Mongoose document - called when you want to save changes to an existing document or a manually created instance.

- > `User.create({ email })` is shorthand for: `const user = new User({ email }); await user.save();`
- > `.save()` runs pre-save middleware (bcrypt hook) - so does `.create()` since it calls `save()` internally.
- > `findOneAndUpdate()` does NOT trigger pre-save hooks (it goes directly to MongoDB). If you need hooks to run, use `find()` then `save()`.

Q6: How does Mongoose's `.lean()` method affect query results? When should you use it?

Answer:

By default, Mongoose queries return Mongoose Document objects - JavaScript objects with prototype methods (`save()`, `validate()`, `get()`, `set()`), virtuals, and tracking for modified paths. `.lean()` returns plain JavaScript objects without any Mongoose-specific overhead.

- > `.lean()` use cases: Read-only operations where you will not call `save()`, API responses where you just need the data, aggregation results (aggregation already returns plain objects).
- > Performance: `.lean()` is 2-3x faster and uses less memory than regular Mongoose Documents because there is no document instantiation overhead.
- > Do NOT use `.lean()` if: You need to call `doc.save()`, `doc.comparePassword()`, or access schema virtuals like `user.fullName`.

Q7: What is a MongoDB aggregation pipeline? How is it different from a regular query?

Answer:

Regular queries (`find`, `findOne`, `findById`) filter and return documents from a single collection with minimal transformation. Aggregation pipelines process documents through a series of stages that can: filter (`$match`), group (`$group`), join (`$lookup`), reshape (`$project`), sort (`$sort`), and compute (`$addFields`, `$count`, `$sum`).

- > Aggregation is more powerful: Can compute statistics (average views per subject), join across collections, reshape output structure, and perform multi-step transformations.
- > Aggregation returns plain JavaScript objects, NOT Mongoose Documents. Schema virtuals, methods, and `instanceof` checks do not work on aggregation output.
- > Use cases in EduStack: Resource count per subject, payment totals per user, most-viewed resources, user enrollment statistics.

Q8: Explain MongoDB's `$match` stage. When should it be the FIRST stage?

Answer:

`$match` filters documents passing through the pipeline, similar to `find()`. It should almost ALWAYS be the first stage because: (1) MongoDB can use indexes for `$match` at the pipeline start, (2) It reduces the number of documents processed by subsequent stages (expensive `$lookup`, `$group`) - critical for performance.

- > Placing `$match` FIRST: MongoDB processes only the matching documents through all subsequent stages.
- > Placing `$match` AFTER `$lookup`: MongoDB first joins ALL documents, then filters. This means joining potentially millions of documents before discarding most of them.
- > Exception: Sometimes you need to `$match` on a joined field. In that case, `$lookup` first then `$match` on the joined field - unavoidable but be aware of the performance cost.

Q9: What is an ObjectId in MongoDB? How is it structured?

Answer:

ObjectId is MongoDB's default `_id` type - a 12-byte unique identifier. Structure: 4 bytes = Unix timestamp (seconds since epoch), 5 bytes = random value (unique per machine+process), 3 bytes = incrementing counter (unique within same second on same machine).

- > ObjectId encodes creation time: `new mongoose.Types.ObjectId().getTimestamp()` returns the date the ID was created. This means you can sort by `_id` to get chronological order.
- > ObjectId vs UUID: ObjectId is 12 bytes (vs UUID's 16 bytes). ObjectId has temporal locality - IDs created together are numerically close, improving B-Tree index performance.
- > Always use ObjectId for `_id` unless you have a specific reason to override. Never use email as `_id` - it cannot be changed without updating all references.

Q10: What is the EduStack DSA sheet caching strategy? What are its limitations?

Answer:

EduStack uses an in-memory TTL cache stored in module-level variables in app.js: `_dsaSheetCache` (parsed problems array) and `_dsaSheetCacheTime` (last fetch timestamp). Cache TTL is 5 minutes. On cache miss, it fetches live from Google Sheets CSV, parses, updates cache, and persists to disk. Fallback: stale cache -> disk file -> empty array.

-> Limitation 1: Not shared across processes. In cluster mode (multiple Node.js instances), each instance has its own cache - they each make independent fetches.

-> Limitation 2: Cache resets on every process restart (Render.com cold starts). The disk fallback (parsed_problems.json) provides continuity.

-> Limitation 3: No cache eviction on update. If the Google Sheet is updated, users see stale data for up to 5 minutes. The `?bust=true` query parameter forces a fresh fetch for admins.

-> Production improvement: Use Redis as shared cache with automatic expiry.

Q11: What is a TTL index in MongoDB? How does it work for OTP collection?

Answer:

A TTL (Time-To-Live) index tells MongoDB to automatically delete documents after a specified time. For the OTP collection, the TTL index is on the `expiresAt` field with `expireAfterSeconds: 0` - meaning MongoDB deletes documents as soon as their `expiresAt` timestamp is in the past.

-> MongoDB's background TTL monitor thread runs every 60 seconds. It scans all TTL-indexed collections and deletes expired documents.

-> No cron job needed: MongoDB handles OTP cleanup automatically. This is more reliable than manual cleanup (no missed runs, no maintenance burden).

-> Behavior when `expiresAt` is reached: The document is NOT immediately deleted - it is deleted within 60 seconds of expiry (TTL monitor granularity). Always check `expiresAt` manually in your code for exact timing.

Q12: Explain the compound index { subject: 1, type: 1 } on the Resource collection.

Answer:

This compound index optimizes queries that filter by both subject AND type. The 1 means ascending order. The index keys are stored sorted: first by subject `ObjectId`, then by type within the same subject. This is the most common query pattern for EduStack's resource API: "give me all notes (type: note) for subject X".

-> Left-prefix rule: { subject: 1, type: 1 } also optimizes queries filtering ONLY by subject (`Resource.find({ subject: id })`). But NOT queries filtering only by type.

-> Without this index: MongoDB must scan ALL resources and check both conditions on each document - $O(n)$ where n is total resources.

-> With this index: MongoDB jumps directly to the subject position in the index, then scans the type sub-range - $O(\log n)$ effectively.

Q13: What is populate() in Mongoose and what queries does it run?

Answer:

`populate()` resolves `ObjectId` references by fetching the referenced documents from their collection. It runs a SEPARATE query for each referenced collection (N+1 query problem for naive use, but Mongoose batches by field).

-> `Resource.find().populate("subject")` runs: 1) `Resource.find()` 2) `Subject.find({ _id: { $in: [all subject ids] } })`. Mongoose merges results in memory.

-> N+1 Problem: If you have 100 resources each referencing different users, `populate("uploadedBy")` makes $1 + 1 = 2$ queries (Mongoose batches all user IDs into one query). NOT 101 queries.

-> Alternative: Use `$lookup` in aggregation for complex joins that need additional pipeline stages (filtering, sorting the joined data).

Q14: How does MongoDB connection pooling work? How many connections does EduStack maintain?

Answer:

Mongoose (via the MongoDB driver) maintains a connection pool - a set of reusable database connections. Rather than opening a new TCP connection for every query (expensive), the pool keeps N connections open and assigns them to queries as needed.

-> Default pool size: Mongoose/MongoDB driver default is 5 connections per pool (configurable via `mongoose.connect(uri, { maxPoolSize: 10 })`).

-> Connection lifecycle: `mongoose.connect()` creates the pool on startup. All controllers share these 5 connections. If all 5 are busy and a 6th query comes in, it waits for a connection to become free.

-> EduStack's bootstrap: app.js connects `mongoose.connect(DB_PATH)` once at startup. All controllers use the same pool. Never call `mongoose.connect()` per request!

Q15: What is the difference between `findById` and `findOne({ _id: id })`?

Answer:

`findById(id)` is Mongoose syntactic sugar for `findOne({ _id: id })`. Mongoose casts the `id` to `ObjectId` automatically. The performance is identical - both use the `_id` index.

-> `findById("abc")` where "abc" is not a valid `ObjectId`: `findById` silently returns null (Mongoose catches the `CastError`).

`findOne({ _id: "abc" })` throws a `CastError` - you must handle it.

-> `findById` is preferred for clarity when fetching by primary key. `findOne` is preferred for compound conditions: `findOne({ email: normalizedEmail })`.

-> `findByIdAndUpdate`, `findByIdAndDelete` are also sugar for `findOneAndUpdate/findOneAndDelete` with `_id`.

Q16: Explain Mongoose virtuals. What is the `fullName` virtual in `EduStack`?

Answer:

Virtuals are computed properties on Mongoose documents that are NOT stored in MongoDB. They are computed on-the-fly when accessed. The `fullName` virtual in `User` model: `userSchema.virtual("fullName").get(function() { return this.firstName + " " + this.lastName; })`

-> Virtuals are NOT included in JSON output by default. To include: `userSchema.set("toJSON", { virtuals: true })` or when calling `doc.toJSON({ virtuals: true })`.

-> Virtual setter: Can create setter virtuals that split a full name into `firstName` + `lastName`.

-> Use cases: computed display names, derived URLs, age from birthdate, aggregated counts from related collections.

Q17: What is `mongoose.models.User || mongoose.model("User", userSchema)`? Why the OR?

Answer:

`mongoose.model("Model", schema)` compiles the model and registers it with Mongoose. If you call `mongoose.model("User", schema)` a SECOND time (e.g., in hot-reload dev environments like nodemon, or when a test file imports the model multiple times), Mongoose throws `OverwriteModelError`.

-> `mongoose.models.User`: If the model is already compiled and registered, return the existing model. If not, compile from schema.

-> In development with nodemon: File changes trigger process restart - model is fresh. No issue.

-> In test environments: Multiple test files may import `User` model. Without this guard, model compilation runs multiple times.

Q18: What does `timestamps: true` do in Mongoose?

Answer:

`timestamps: true` automatically adds two fields to every document: `createdAt` (Date - when the document was first created) and `updatedAt` (Date - automatically updated whenever the document is saved). These are managed by Mongoose, not by your code.

-> Performance: `createdAt` is useful for sorting by newest first: `User.find().sort({ createdAt: -1 })`

-> `updatedAt` is automatically updated on every `.save()`, `findOneAndUpdate()`, etc. - you never need to manually set it.

-> `EduStack` uses `timestamps: true` on ALL schemas (`User`, `Subject`, `Resource`, `Payment`, `OTP`) for auditing and debugging.

Q19: What is the `upsert` option in Mongoose? How does `EduStack` use it for `OTP`?

Answer:

`upsert: true` in `findOneAndUpdate` means: if a matching document is found, UPDATE it; if NO matching document is found, CREATE a new one (insert). This is an atomic operation - find + create/update in a single MongoDB command.

-> `OTP upsert`: `OTP.findOneAndUpdate({ email }, { code, expiresAt }, { upsert: true })`. First `OTP` request: creates new `OTP` document. Resend `OTP`: updates the existing document with a new code and expiry.

-> Without `upsert`: You would need `findOne()` then if (existing) `update()` else `create()` - two round-trips and a potential race condition.

-> `upsert` is also used for: session management, counter updates (but use `$inc` for counters), idempotent operations.

Q20: What is `select: false` in Mongoose? Name other field-level options.

Answer:

`select: false` means the field is EXCLUDED from all query results unless explicitly requested with `.select("+fieldName")`. Used in `EduStack` for password - prevents accidental exposure.

-> Other field options: `required: [true, "Error msg"]` (validation). `unique: true` (unique index). `default: value` (default value). `min/max` (numeric range validation). `maxlength` (string length validation). `trim: true` (auto-trim whitespace). `lowercase: true` (auto-lowercase). `enum: [...]` (only allowed values). `index: true` (create index). `match: [regex, "msg"]` (regex validation).

-> Type coercion: Mongoose automatically casts values to the declared type. "3" for a `Number` type becomes 3. "true" for `Boolean` becomes true.

-> Custom validators: validate: { validator: fn, message: "msg" } for complex validation rules.

Q21: Explain write concern in MongoDB. What is the default?

Answer:

Write concern specifies how many replica set members must acknowledge a write before the MongoDB driver considers it successful. Default: { w: 1 } - the primary node acknowledges.

- > { w: 0 }: No acknowledgment - fire-and-forget. Fastest but data can be lost.
- > { w: 1 }: Primary acknowledges. Standard for most operations.
- > { w: "majority" }: More than half the replica set acknowledges. Survives primary failure. Recommended for critical data (EduStack payments, user creation).
- > { j: true }: Primary acknowledges AND confirms journal write. Guarantees durability even if process crashes.

Q22: What is MongoDB Atlas? How does EduStack use it?

Answer:

MongoDB Atlas is MongoDB's fully managed cloud database service. It provides: automatic backups, point-in-time recovery, automatic failover with replica sets, performance monitoring, automatic security patching, and a visual query performance advisor.

- > EduStack connects via: mongoose.connect(process.env.MONGO_URI) where MONGO_URI is the Atlas connection string (mongodb+srv://...). The connection string includes credentials, cluster hostname, and TLS settings.
- > Atlas features EduStack benefits from: Replica set (automatic failover), Atlas Search (could enable full-text search on subjects), Performance Advisor (suggests missing indexes), Data Explorer (visual DB browser).
- > MONGO_URI is in .env - never hardcoded. The Atlas cluster uses TLS encryption for all connections.

Q23: What is the difference between embedded documents and references in MongoDB?

Answer:

Embedded: Store related data directly inside the parent document as a subdocument or array. References: Store the ObjectId of the related document and use populate() or \$lookup to retrieve it.

- > Embed when: Related data is always accessed together, data belongs to one parent, no need to query related data independently. Example: User's address object embedded in user document.
- > Reference when: Related data is large and not always needed, data has its own identity and can be updated independently, many-to-many relationships. EduStack: Resource references Subject by ObjectId (a resource belongs to one subject, but subjects can be queried independently).
- > EduStack design: Resources reference Subject (many resources per subject - reference). Subject does NOT embed resources (too many, and resources have their own lifecycle).

Q24: What is the aggregation \$group stage? What accumulators are available?

Answer:

\$group groups documents by a specified key (_id field) and computes aggregate values for each group using accumulator operators.

- > Accumulators: \$sum (total or count), \$avg (average), \$min/\$max (min/max value in group), \$push (array of values), \$addToSet (unique values array), \$first/\$last (first/last value in group after sorting), \$count (number of documents).
- > Example: Group resources by type and count: { \$group: { _id: "\$type", count: { \$sum: 1 } } } - gives { _id: "note", count: 45 }, { _id: "pyq", count: 23 }, etc.
- > Important: \$group can only compute values from fields in the documents entering the stage. To include subject name, you need \$lookup before \$group.

Q25: How does EduStack parse and use the Google Sheets CSV data?

Answer:

EduStack fetches the published CSV URL from Google Sheets using Node's built-in https.get() (not axios/fetch), following HTTP redirects manually. parseCSVText() manually parses CSV text handling quoted fields, escaped quotes, and CRLF line endings. csvLinesToProblems() maps CSV rows to problem objects.

- > Custom CSV parser: Handles edge cases like fields containing commas in quotes: "Google, Amazon" is a single field. Standard JSON.parse() cannot handle this.
- > Google Sheets redirect: The CSV URL redirects (302) to the actual data URL. EduStack's fetchCSV() follows redirect chains manually.
- > Merge with static file: Live CSV data is merged with parsed_problems.json to preserve manually-added GitHub solution URLs and video links that are not in the Google Sheet.

Q26: What is connection string format in MongoDB? What is the difference between mongodb:// and

mongodb+srv://?

Answer:

mongodb://: Legacy connection string. Specifies host:port directly. Example: mongodb://user:pass@localhost:27017/mydb.

-> mongodb+srv://: DNS Seedlist connection format. Instead of listing all replica set members, uses a single SRV DNS record that resolves to multiple hosts. MongoDB Atlas uses this format.

-> SRV advantages: When Atlas adds/removes nodes from the replica set, the DNS record is updated - no need to update connection strings. Also includes TLS settings automatically.

-> EduStack: Uses mongodb+srv://username:password@cluster.mongodb.net/edustack - Atlas format with automatic TLS.

Q27: Explain Mongoose's populate() with select. How do you avoid over-fetching?

Answer:

populate("field") fetches ALL fields of the referenced document by default. Use populate({ path: "field", select: "name email - _id" }) to select only needed fields.

-> Over-fetching problem: Resource.find().populate("subject") fetches the entire Subject document (description, thumbnail, all fields) even if you only need subject.name.

-> Optimized: Resource.find().populate("subject", "name code - _id") - only fetches name and code from Subject. Reduces network bandwidth and memory.

-> EduStack example: Resource.find().populate("uploadedBy", "firstName email") - only fetches first name and email from the uploading admin user.

Q28: What is an aggregation \$project stage? How does it differ from .select()?

Answer:

\$project shapes the output of a pipeline stage - include or exclude fields, rename fields, compute new fields using expressions. Unlike .select() (which can only include/exclude existing fields), \$project can compute new fields using operators (\$concat, \$sum, \$cond, \$dateToString, etc.).

-> Include fields: { \$project: { name: 1, code: 1 } } - include only name and code.

-> Compute new fields: { \$project: { fullName: { \$concat: ["\$firstName", " ", "\$lastName"] }, yearJoined: { \$year: "\$createdAt" } } }

-> Rename fields: { \$project: { subjectName: "\$name" } } - renames "name" to "subjectName" in output.

Q29: How does EduStack handle the case where Cloudinary upload fails?

Answer:

If cloudinary.uploader.upload() throws an error (network timeout, quota exceeded, invalid image), EduStack catches the error and falls back to storing the image as a base64 data URI string directly in the User.avatar field.

-> Fallback: avatarUrl = "data:image/jpeg;base64," + req.file.buffer.toString("base64") - stores the entire image binary as a string in MongoDB.

-> Limitation: Base64 encoding increases size by ~33%. A 100KB image becomes a 133KB MongoDB field. Large base64 data URIs slow down user document reads.

-> Better production approach: If Cloudinary fails, either retry, use a fallback CDN, or return an error and ask the user to upload again. Storing binary in MongoDB is an antipattern for images.

Q30: What is mongoose.connect()? What are the important options?

Answer:

mongoose.connect(uri, options) establishes the MongoDB connection pool and returns a Promise. EduStack calls it once at server startup in app.js before starting the HTTP server.

-> Key options: maxPoolSize (default 5), serverSelectionTimeoutMS (how long to wait to find a healthy server), socketTimeoutMS (how long to wait for a response), family: 4 (force IPv4 - sometimes needed in cloud environments).

-> Error handling: EduStack uses .then() and .catch(). If connection fails, process.exit(1) is called - no point running without a database.

-> Mongoose events: mongoose.connection.on("connected", ...), mongoose.connection.on("error", ...) - useful for monitoring.

Q31: What is the \$lookup aggregation operator? Write an example.

Answer:

\$lookup is MongoDB's aggregation JOIN. It combines documents from two collections based on a matching field. Produces an array of matching documents in the output.

-> Basic \$lookup: { \$lookup: { from: "users", localField: "uploadedBy", foreignField: "_id", as: "uploaderInfo" } } - joins resources with their uploading user.

-> \$unwind: Since \$lookup produces an array, \$unwind flattens it: { \$unwind: "\$uploaderInfo" } converts the array to a single embedded object.

-> Pipeline \$lookup (more powerful): { \$lookup: { from: "resources", let: { subId: "\$_id" }, pipeline: [{ \$match: { \$expr: { \$eq: ["\$subject", "\$\$subId"] } } }, { \$limit: 5 }], as: "resources" } } - join with additional pipeline stages inside lookup.

Q32: What is the N+1 query problem in Mongoose? How do you solve it?

Answer:

N+1 problem: Fetching N parent documents, then making a separate query for each parent's related data - N+1 total queries. Example: Fetch 20 resources (1 query), then for each resource fetch its subject (20 queries) = 21 total queries.

-> Mongoose solution: populate() batches all related IDs into a single \$in query: 1 query for resources + 1 query for ALL unique subjects = 2 queries (not 21).

-> \$lookup solution: Single aggregation pipeline - 1 query total for both resources and their subjects.

-> DataLoader pattern (for GraphQL): Batches and caches related queries. Not needed for EduStack's REST API since Mongoose populate() handles batching automatically.

Q33: Explain how EduStack's CSVLinesToProblems function works at a high level.

Answer:

csvLinesToProblems() processes the 2D array of CSV rows (lines) and converts them into structured problem objects. It maintains state for the "current category" and "current subtopic" as it iterates rows. Special rows update the category/subtopic state. Data rows (where col2 is a number) are converted to problem objects.

-> Category detection: If col0 (first cell) has a non-empty, non-admin value, it becomes the new currentCategory.

-> Subtopic detection: Rows where col2 is not a number AND col4 is empty/placeholder are treated as subtopic headers.

-> Company name filtering: isOnlyCompanyNames() checks if an "intuition" field is actually just company names (e.g., "Google, Amazon") - filters these out to prevent misleading data.

Q34: What is a MongoDB replica set? How does it provide high availability?

Answer:

A replica set is a group of MongoDB servers that maintain the same data. One server is the Primary (handles all writes). Others are Secondaries (replicate from primary asynchronously). If the primary fails, secondaries hold an automatic election to choose a new primary.

-> Atlas M0 (free tier): 3-node replica set in a single region. Auto-failover within ~30 seconds if primary fails.

-> Replication: Every write to the primary is replicated to secondaries within milliseconds. Read preference can route reads to secondaries to distribute load.

-> Connection string: mongoose.connect("mongodb+srv://...") automatically discovers all replica set members via DNS SRV records. Client transparently reconnects to the new primary after failover.

Q35: What is the significance of the "sessions" collection created by connect-mongodb-session?

Answer:

The sessions collection stores user session data for EduStack's Google OAuth flow. Each document represents one active session: { _id: sessionId, session: { passport: { user: userId }, expires: Date }. The _id is the session ID stored as a cookie in the user's browser.

-> Session ID security: The session ID is a cryptographically random string (not predictable). It is signed with JWT_SECRET using the session secret option - prevents session ID forgery.

-> TTL: sessions collection has a MongoDB TTL index on expires field. Expired sessions are auto-deleted. No manual session cleanup needed.

-> Session vs JWT trade-off: If the sessions collection is deleted, all logged-in users are logged out. JWT has no central store - you cannot invalidate JWTs this way.

Q36: How does mongoose.find() differ from mongoose.aggregate()? When to use each?

Answer:

find() returns Mongoose Documents from a single collection with optional filtering, projection, sorting, and population. aggregate() processes documents through a pipeline of stages that can transform, join, group, and compute - returns plain JavaScript objects.

-> Use find() when: Simple CRUD queries, need Mongoose Document features (save(), populate(), virtuals), single collection access.

-> Use aggregate() when: Computing statistics, grouping data, joining multiple collections in complex ways, reshaping output structure, counting with complex conditions.

-> Performance: aggregate() can be more efficient for complex operations (server-side processing) but is more complex to write. find() is simpler for straightforward queries.

Q37: What is the difference between `mongoose.Schema.Types.ObjectId` and `String` for ref fields?

Answer:

Using `mongoose.Schema.Types.ObjectId` for ref fields (like `subject`, `uploadedBy`, `user`) is strongly preferred over `String` because: (1) Mongoose validates that the value is a valid `ObjectId` format before saving, (2) `populate()` works correctly with `ObjectId` refs, (3) `$lookup` and `$match` with `ObjectIds` use the indexed `_id` field.

-> If you use `String` for a ref field and store the `ObjectId` as a string, `populate()` still works but `$match` comparisons with `$lookup` may fail if the types don't match (`String` vs `ObjectId`).

-> Always declare ref fields as: `{ type: mongoose.Schema.Types.ObjectId, ref: "modelName" }`.

-> Mongoose auto-casts: `String "64f7a2b3c8..."` passed to an `ObjectId` field is automatically cast to `ObjectId` by Mongoose - you don't need to do `mongoose.Types.ObjectId("...")` manually.

Q38: Explain the Cloudinary folder structure EduStack uses. Why have separate folders?

Answer:

EduStack uses two Cloudinary folders: `"edustack_profiles"` for user avatar images and `"edustack_subjects"` for subject thumbnail images. Separate folders provide: organized storage, easy bulk management (delete all subject thumbnails), separate access policies, and clear purpose identification.

-> Cloudinary folder: `{ folder: "edustack_profiles" }` in upload options. The uploaded file URL becomes: `https://res.cloudinary.com/yourcloud/image/upload/v1234/edustack_profiles/filename.jpg`

-> Transformations: Cloudinary can auto-resize images in the URL. Requesting `.../w_200,h_200,c_fill/...` crops the avatar to 200x200. This eliminates the need for server-side image processing.

-> CDN: Cloudinary's global CDN serves images from the nearest edge to the requesting user - faster than serving from Render.com's single server location.

Q39: What happens when `mongoose.connect()` fails? How does EduStack handle it?

Answer:

If `mongoose.connect()` rejects (wrong credentials, network issue, Atlas cluster paused), EduStack's `.catch(err => { console.error(err.message); process.exit(1); })` immediately exits the Node.js process with error code 1.

-> Why exit instead of retry? EduStack has no database retry logic. The process manager (Render.com) will restart the process automatically. On Render.com, a failed start causes the deploy to show as "failed" - visible in the dashboard.

-> `serverSelectionTimeoutMS`: Controls how long Mongoose waits before giving up on finding a MongoDB server. Default is 30 seconds - long for a startup timeout.

-> Production improvement: Add a retry loop with exponential backoff: try connecting up to N times with increasing delays before giving up.

Q40: What is the `.env` file? What secrets does EduStack store in it?

Answer:

The `.env` file stores environment-specific configuration values, especially secrets that must not be committed to version control. EduStack reads it with `require("dotenv").config()` which loads variables into `process.env`.

-> EduStack `.env` secrets: `MONGO_URI` (DB connection with credentials), `JWT_SECRET` (signs JWTs), `GOOGLE_CLIENT_ID/SECRET` (OAuth credentials), `RAZORPAY_KEY_ID/SECRET` (payment API keys), `CLOUDINARY_CLOUD_NAME/API_KEY/API_SECRET` (CDN credentials), `EMAIL_USER/PASS` (SMTP credentials), `ADMIN_EMAILS` (admin email list).

-> Security: `.env` is in `.gitignore` - never committed. A `.env.example` with placeholder values (`MONGO_URI=mongodb+srv://USERNAME:PASSWORD@cluster/db`) documents required variables without exposing secrets.

-> Render.com: Environment variables are set via the Render dashboard Environment tab - they become `process.env.VAR_NAME` in the running process.